WEB SCRAPING PROJECT REPORT

Project Title: Web Scraping and Dataset Creation Using Python
Student Name: Raiyan Abdullah
Course: Computer Science
Date: 19 October 2026

1. Introduction

Web scraping is the process of automatically collecting information from websites using computer programs. It helps researchers and analysts gather structured data from webpages for further analysis.

This project demonstrates how Python can be used to extract book information from the Books to Scrape website. The collected information includes book titles, prices, ratings, availability, and webpage URLs. The data was organized into a structured dataset and analyzed to identify basic patterns in book prices and ratings.

2. Aim and Objectives

The aim of this project was to develop a working Python web scraper that collects information from a public practice website and stores it in a dataset for analysis.

The objectives were to:

1. Connect to a website using Python.
2. Retrieve and parse webpage HTML.
3. Extract relevant book information.
4. Collect data from multiple pages.
5. Store the collected information in a CSV file.
6. check data quality and identify missing values.
7. Perform basic statistical analysis on the collected data.

3. Tools and Technologies

The following tools and Python libraries were used:

- **Google Colab:** An online environment for writing and executing Python code.
- **Python:** The programming language used to develop the scraper.
- **Requests:** Used to send HTTP requests and retrieve webpage content.
- **BeautifulSoup:** Used to parse HTML and extract book information.
- **Pandas:** Used to organize, clean, analyze, and export the collected data.

- **CSV:** Used as the output format for the dataset.

4. Website and Data Collection

The project used the Books to Scrape website, a practice website designed for learning web scraping.

Website: https://books.toscrape.com/

The scraper sent HTTP requests to the website and received its HTML content. The Requests library returned a successful HTTP status code of 200 during the connection test.

BeautifulSoup was then used to locate the HTML elements containing the book information. The scraper collected data from five catalogue pages, with a target of 20 books per page.

The following fields were collected:

Field	Description
Title	Name of the book
Price	Displayed book price
Rating	Book rating, from one to five stars
Availability	Stock availability shown on the webpage
Book_URL	Link to the individual book page
Price_numeric	Numeric version of the price for calculations

The scraped information was stored in a Pandas DataFrame and exported to a CSV file named books_dataset.csv.

5. Methodology

The project was completed using the following steps:

Step 1: Library installation and testing

The required Python libraries were installed or verified in Google Colab. Import tests confirmed that Requests, BeautifulSoup, and Pandas were available.

Step 2: Website connection

A request was sent to the Books to Scrape homepage. The returned HTTP status code was 200, confirming that the webpage was retrieved successfully.

Step 3: HTML parsing

BeautifulSoup parsed the webpage HTML. CSS selectors were used to identify book listing elements and extract titles, prices, ratings, and availability.

Step 4: Multi-page scraping

The scraper visited five catalogue pages and collected book records from each page. It followed the website's pagination links and included a short delay between requests.

Step 5: Data preparation

The collected records were organized into a Pandas DataFrame. Duplicate titles were removed, and prices were converted into numeric values for statistical calculations.

Step 6: Data export

The final dataset was saved as books_dataset.csv using UTF-8 encoding.

Step 7: Data analysis

Pandas was used to calculate the average, minimum, and maximum book prices and count the number of books in each rating category.

6. Results and Findings

The scraper successfully processed five catalogue pages and collected 100 unique book records.

6.1 Summary of Results

Measure	**Result**
Catalogue pages scraped	5
Unique book records	100
Missing values in reported columns	0
Average book price	£34.30
Lowest book price	£10.16
Highest book price	£58.11

The results show that the lowest-priced book in the analyzed dataset was *Patience*, priced at £10.16. The highest book price was £58.11.

6.2 Distribution of Book Ratings

Rating	**Number of books**

One star	22
Two stars	19
Three stars	22
Four stars	18
Five stars	19
Total	**100**

The one-star and three-star categories each contained 22 books, making them the most frequent rating categories in this sample. The four-star category contained 18 books, the lowest count among the five categories.

6.3 Data Quality

The missing-value check reported zero missing values in each of the six dataset columns. The scraper also removed duplicate records based on book titles.

The numeric price field made it possible to calculate summary statistics and sort the books by price. The CSV output provides a reusable dataset that can be opened in spreadsheet applications or analyzed further with Python.

7. Challenges and Solutions

Several technical considerations arose during the project.

HTML structure: Extracting information required identifying the correct HTML elements and CSS selectors. BeautifulSoup helped locate the relevant elements.

Text encoding: The displayed currency symbol initially appeared incorrectly in the printed output. The text was normalized during dataset preparation so that prices displayed correctly.

Pagination: Collecting more than one page required following the website's pagination links. The scraper was designed to process five pages and stop if a next-page link was unavailable.

Data quality: Missing-value and duplicate checks were performed to improve the reliability of the dataset.

8. Ethical Considerations

The project used a practice website intended for learning web scraping. Requests were made sequentially, with a short delay between page requests.

Web scraping should be performed responsibly. Before collecting data from other websites, users should review the site's terms of service, applicable policies, and access restrictions.

Requests should not overload a server, bypass authentication, or collect private or restricted information.

9. Limitations

The dataset represents only five catalogue pages and is not a complete collection of every book on the website. The analysis describes the collected sample rather than the entire catalogue or the wider book market.

The project collected the information displayed on the website at the time of scraping. Prices, availability, and other details may change. The results therefore represent a snapshot of the collected data.

10. Conclusion

This project demonstrated how Python can be used to scrape, organize, and analyze information from a website. Using Requests and BeautifulSoup, the program retrieved webpage content and extracted book information. Pandas was then used to prepare the data, check for missing values, calculate price statistics, and summarize ratings.

The completed scraper collected 100 unique book records across five catalogue pages. The dataset contained no reported missing values, had an average book price of £34.30, and showed prices ranging from £10.16 to £58.11.

The project achieved its main objectives and provided practical experience in web scraping, HTML parsing, data cleaning, CSV export, and exploratory data analysis.

11. Recommendations

Future improvements could include:

1. Scraping all catalogue pages instead of limiting the project to five pages.
2. Adding error handling for failed requests and unexpected HTML changes.
3. Including additional fields, such as book categories or availability quantities, where available.
4. Creating charts to visualize price distributions and rating frequencies.
5. Scheduling periodic data collection where permitted, to monitor changes over time.

12. References

1. Books to Scrape. https://books.toscrape.com/
2. Requests documentation. https://requests.readthedocs.io/
3. Beautiful Soup documentation. https://www.crummy.com/software/BeautifulSoup/bs4/doc/

4. Pandas documentation. https://pandas.pydata.org/docs/
5. Google Colab. https://colab.research.google.com/

Appendix: Project Evidence

The following evidence should accompany the report:

- Screenshot of the successful website connection showing HTTP status code 200.
- Screenshot of the first-page book extraction.
- Screenshot of the 100-record dataset preview.
- Screenshot of the missing-value summary and statistical analysis.
- The Python notebook (.ipynb) and exported CSV dataset (books_dataset.csv).